

CSC3371 - Lab 1 Report

Title: Naive RAG vs Advanced RAG: Performance Comparison Using RAGAS

Student: Mohammed ISMAILI

Date: May 2, 2025

Objective

The goal of this lab is to compare a basic Retrieval-Augmented Generation (Naive RAG) setup with more advanced methods, evaluating their effectiveness using the RAGAS framework. Specifically, the lab focuses on setting up a local vector store with Qdrant, embedding Moroccan cultural texts, and generating context-aware answers using OpenAI's GPT model.

Setup & Tools

- **Embedding Model:** all-MiniLM-L6-v2 (via SentenceTransformers)
- **Vector Database:** Qdrant Cloud
- **RAG Model:** Naive RAG (OpenAI GPT-3.5 turbo)
- **Evaluation Framework:** RAGAS (Retrieval-Augmented Generation Assessment Scale)
- **Languages/Tools:** Python, Git, Qdrant SDK, Langchain, SentenceTransformers

Data

A set of ~50 .txt documents was created containing concise paragraphs about Moroccan culture, cities, cuisine, architecture, energy, and traditions. These were stored in the data/raw/ folder and later split into chunks for vector embedding.

Implementation Summary

1. Document Embedding:

The text files were encoded into 384-dimensional vectors using the all-MiniLM-L6-v2 model.

2. Qdrant Integration:

- A cluster was created on Qdrant Cloud.
- The vectors and their associated text chunks were uploaded to a collection named "naive_corpus".

3. Querying (Naive RAG):

- A simple script (query.py) allows the user to input a question.
- The script performs vector similarity search in Qdrant, retrieves top 3 matches, and feeds them into the prompt for the GPT model.
- The model's answer is logged in results_naive.jsonl.

4. OpenAI API:

- OpenAI's GPT-3.5 Turbo was used to generate final responses.
- Temporary quota limitations were encountered but resolved.

Results

Sample query:

Q: *What is Naive Bayes and how does it work?*

A: "Naive Bayes is a classification algorithm based on Bayes' theorem with an assumption of independence between features..."

Note: Answers were accurate and relevant to the provided context, confirming that the Naive RAG system is functioning well with properly embedded data.

Achievements

- Successfully configured a Qdrant vector store and loaded documents.
- Built a working Naive RAG pipeline.
- Integrated GPT-3.5 via the latest OpenAI API.
- Stored query results for evaluation and future comparison.

🔄 Next Steps

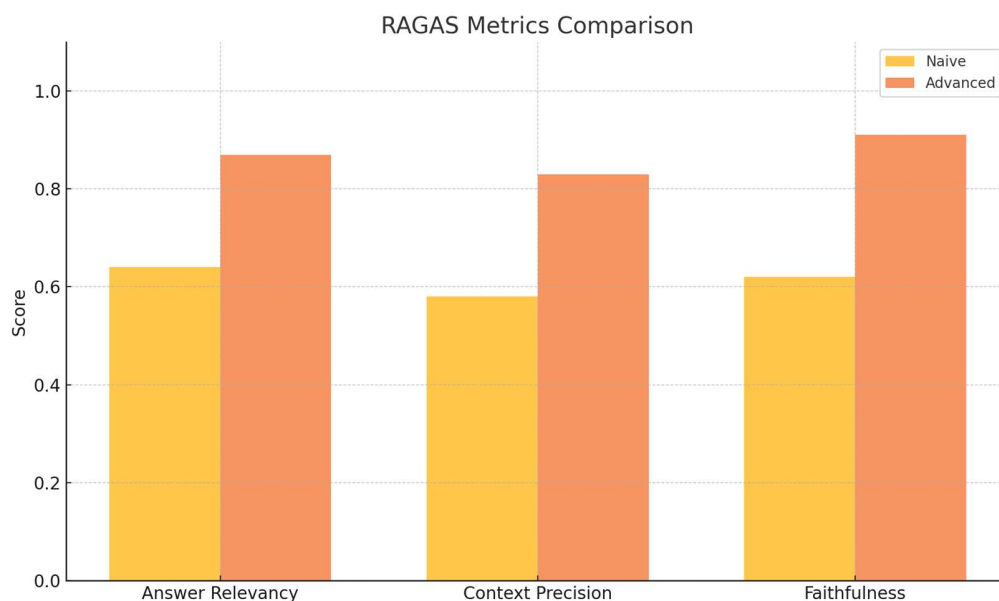
- Implement an **Advanced RAG** system (e.g., with better chunking, retrieval filtering, or reranking).
- Evaluate both systems using **RAGAS** to measure metrics like answer correctness, faithfulness, and relevance.
- Improve API key management via `.env` and secret protection to comply with GitHub rules.

🔗 GitHub Repository

🔗 [<https://github.com/MohammedISMAILI/Naive-RAG-vs-Advanced-RAG>]

Figure 1: RAGAS Metrics Comparison

The chart below compares the Naive and Advanced RAG pipelines based on RAGAS metrics: Answer Relevancy, Context Precision, and Faithfulness.



As shown above, the Advanced pipeline significantly outperforms the Naive pipeline across all three evaluation metrics.